

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая/Лабораторная работа

Выполнил:

Бочарников Максим

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

1. Цель работы

Освоить полный цикл разработки backend-приложения с реляционной базой данных на реальном проекте: проектирование схемы предметной области, контейнеризация инфраструктуры, применение миграций, инициализация данных, реализация REST API, настройка документации и базовой проверки работоспособности сервиса.

2. Задачи

- Спроектировать модель данных для платформы обработки и классификации анкетных ответов (пользователи, роли, компании, проекты, исследования, вопросы, категории, классификации, тарифы, подписки, файлы и фоновые задания).
- Развернуть PostgreSQL и Redis в Docker и связать backend с окружением через .env.
- Описать и поддерживать схему БД через SQLAlchemy + Alembic.
- Реализовать API на FastAPI с версионированием /api/v1.
- Настроить аутентификацию и авторизацию (login/refresh/logout, роль пользователя, проверка активной подписки).
- Подключить OpenAPI-документацию.
- Подготовить сценарии проверки API (Swagger UI + автотесты pytest для ключевых эндпоинтов).

3. Краткое описание реализованного сервиса

Реализован backend системы Tagmentor для работы с опросами и AI-классификацией:

- Пользователь принадлежит компании и связан с ролями (admin, manager, user) через таблицу user_roles.
- Компания работает по тарифам и подпискам (tariffs, subscriptions), что влияет на доступ к функциональности.
- Внутри компании создаются проекты (projects), в проектах — исследования (surveys).
- Исследования содержат вопросы (questions), категории (categories) и классификации ответов (classifications), включая связь M:N categories_classifications.

- Файлы исследования (files) используются для предпросмотра/генерации и асинхронной обработки (job_files).
- Учитывается потребление AI-моделей через сущность ai_answers (входные/выходные токены по тарифу и проекту).

4. Архитектурные и технологические решения

- Backend: FastAPI, запуск через uvicorn (debug) и gunicorn (production).
- Данные: PostgreSQL 14, ORM/миграции — SQLAlchemy 2 + Alembic.
- Кэш/служебное хранилище: Redis.
- Безопасность: JWT (python-jose), хеширование паролей (argon2/passlib), cookies access_token и refresh_token.
- API-документация: Swagger/OpenAPI (/api/v1/docs, /api/v1/openapi.json), доступ к документации защищен Basic Auth middleware.
- Наблюдаемость: Prometheus, Loki, Promtail, Grafana.
- Дополнительный сервис: отдельный контейнер ml для AI-задач и интеграций.

5. Развертывание базы данных и окружения

5.1 Контейнеризация

В docker-compose.yml описаны сервисы:

- tagmentor-postgres (PostgreSQL 14),
- tagmentor-redis,
- tagmentor-backend,
- tagmentor-scheduler,
- tagmentor-frontend,
- tagmentor-ml,
- стек мониторинга (prometheus, loki, promtail, grafana).

Данные PostgreSQL сохраняются в постоянном volume/host path.

5.2 Подключение приложения

Параметры подключения и секреты задаются через `env/*.env` (для `backend`, `postgres`, `redis` и др.).

5.3 Миграции

Схема БД поддерживается через `Alembic`.

При старте `backend` выполняется `alembic upgrade head`, что синхронизирует БД с текущей версией моделей.

5.4 Начальные данные

Инициализация базовых записей реализована в `backend/migrations/insert.py`:

- роли,
- тариф,
- тестовые компании,
- пользователи,
- привязка ролей,
- проект, подписка, тестовые AI-ответы, исследование.

6. Реализация REST API

6.1 Точка входа

Основное приложение собрано в `backend/main.py`, API-модуль монтируется через `backend/apps/api_app.py` на префикс `/api/v1`.

6.2 Основные модули API

Реализованы роутеры:

- auth — /auth/login, /auth/refresh, /auth/logout;
- users — /users/me, /users/all, /users/create, /users/patch;
- companies, projects, surveys, questions, categories, classifications;
- files — предпросмотр/генерация файлов;
- subscriptions, tariffs, roles;
- jobs, stream, callbacks, ai.

6.3 Авторизация и разграничение доступа

- login: валидация пользователя и выдача access/refresh JWT;
- refresh: обновление пары токенов по cookie refresh_token;
- logout: удаление токенов;
- защищенные эндпоинты используют dependency AuthorizationNamespace.authorize;
- часть функционала дополнительно ограничена активной подпиской (active_subscription_required).

6.4 Контракты и обработка ошибок

Для эндпоинтов используются Pydantic-схемы из templates/schemas/*, а ответы стандартизируются через templates/responses.py.

Ошибки возвращаются предсказуемо с соответствующими HTTP-кодами (400/401/403/404/500).

6.5 Документация

После запуска backend доступны:

- Swagger UI: <http://127.0.0.1:8000/api/v1/docs>,
- OpenAPI JSON: <http://127.0.0.1:8000/api/v1/openapi.json>.

7. Порядок запуска проекта (локально)

Вариант без контейнера для backend:

1. Поднять инфраструктуру:
 - `docker compose up --no-recreate -d tagmentor-redis tagmentor-postgres`
2. Установить зависимости backend:
 - `cd backend`
 - `pip install -r requirements.txt`
3. Запустить backend:
 - `py main.py debug`

Контейнерный вариант:

- из корня проекта: `make`

Проверка:

- открыть `/api/v1/docs`,
- выполнить сценарий: `login -> users/me -> users/all -> projects/surveys/files` (в зависимости от роли и подписки).

8. Результаты

- Развернута полноценная инфраструктура backend-приложения в Docker.
- Реализована и применена реляционная схема данных для домена исследований и классификации.
- Настроены аутентификация, авторизация и бизнес-ограничения по подписке.
- Реализован REST API с версионированием `/api/v1`.
- Включены OpenAPI-документация и базовые автотесты (pytest, модуль пользователей).
- Добавлены средства мониторинга и логирования (Prometheus + Grafana + Loki).

9. Выводы

В рамках лабораторной работы отработан полный практический цикл backend-разработки: от модели данных и миграций до рабочего API и эксплуатационного окружения.

Использование FastAPI упростило построение и документирование HTTP-контрактов, а SQLAlchemy + Alembic обеспечили управляемую эволюцию схемы БД.